

- CareerOrbit V3.1 — The Frontend Master Guide
  - 1. Project Overview & Vision
  - 2. Core Technology Stack
  - 3. Global Design System (The "Professional Shadow")
    - Design Philosophy
    - Styling Tokens
  - 4. The Mastery Engine (UI State Mapping)
  - 5. The Cinematic Mind-Map System
    - Coordinate System & Camera Logic
  - 6. Component Library Breakdown
    - A. MindMapNode.tsx
    - B. TechnicalSidePanel.tsx
    - C. StreamingModuleCard.tsx
    - D. MasteryDashboard.tsx
  - 7. Page-by-Page Technical Walkthrough
    - A. /onboarding (Psychometric Discovery)
    - B. /path-selection (Opportunity Engine)
    - C. /pre-test (Baseline Initialization)
    - D. /learn (Adaptive Classroom)
    - E. /dashboard (Mastery Analytics)
    - F. /architecture (Systems Overview)
  - 8. AI & Streaming Integration
  - 9. Development Guidelines

# CareerOrbit V3.1 — The Frontend Master Guide

**Audience:** Frontend Engineering Team **Objective:** To provide a complete, end-to-end technical understanding of the CareerOrbit UI, starting from zero.

## 1. Project Overview & Vision

CareerOrbit is an adaptive workforce development platform. Unlike traditional websites, the UI is **deterministic**—it changes its structure and content based on a "Mastery

Score" (a probability value between 0 and 1) calculated for every user skill. Your goal as a frontend developer is to ensure this adaptive experience feels cinematic, responsive, and professional.

---

## 2. Core Technology Stack

---

- **Framework:** Next.js 14 (App Router).
  - **Language:** TypeScript (Strict Mode).
  - **Styling:** Tailwind CSS + Vanilla CSS (for complex animations).
  - **Motion:** Framer Motion (used for all transitions and canvas movements).
  - **AI Integration:** Vercel AI SDK (for streaming content directly into components).
  - **Database Layer:** Drizzle ORM (interfaced via Server Actions).
- 

## 3. Global Design System (The "Professional Shadow")

---

We use a premium, dark-mode aesthetic. Do not deviate from these tokens.

### Design Philosophy

- **Background:** Pure Black (#000000) for maximum focus and OLED efficiency.
- **Primary Accents:** Emerald Green (SDG 8) and Deep Indigo for success/progress states.
- **Typography:** Modern, high-readability sans-serif (**Outfit** for headings, **Inter** for body).

### Styling Tokens

- **Background:** `bg-[#000000]`
- **Surface Layer (Glassmorphism):**
  - `backdrop-filter: blur(12px) saturate(180%);`
  - `background: rgba(17, 17, 17, 0.7);`
  - `border: 1px solid rgba(255, 255, 255, 0.1);`

- **Animations:** GPU-accelerated transforms and opacity transitions only.
- 

## 4. The Mastery Engine (UI State Mapping)

---

The UI is driven by a `knowledgeState` object. Every skill has a `pMastery` probability ( $pM$ ).

- **Rule:** If  $pM < 0.30$ , show **Locked/Scaffolded** (Grayscale, reduced opacity).
  - **Rule:** If  $0.30 \leq pM < 0.85$ , show **Active/ZPD** (High-contrast, glowing borders).
  - **Rule:** If  $pM \geq 0.85$ , show **Mastered** (Emerald Green accent, verified badge).
- 

## 5. The Cinematic Mind-Map System

---

Implemented in `ArchitectureClient.tsx`, this is a virtualized 2D plane for structural navigation.

### Coordinate System & Camera Logic

- **Camera Offset:** When the technical side-panel opens (right 400px), the camera applies a negative horizontal offset to keep the active node centered in the *visible* workspace:
    - `centerX = (window.innerWidth / 2) - 200px`
  - **Spring Physics:** Camera movements use `transition: { type: "spring", stiffness: 120, damping: 20 }` for a fluid, organic feel.
- 

## 6. Component Library Breakdown

---

### A. `MindMapNode.tsx`

The primary interactive element. Receives  $(x, y)$  coordinates and mastery status. Triggers camera movement and side-panel events.

## B. `TechnicalSidePanel.tsx`

A glassmorphic drawer using `AnimatePresence`. Renders high-depth Markdown (KaTeX supported) for technical documentation.

## C. `StreamingModuleCard.tsx`

Uses Vercel AI SDK's `useObject` to render content as it's generated by the AI in real-time.

## D. `MasteryDashboard.tsx`

The analytics hub. Includes the **Mastery Grid** (heat-map), **Learning Curve** (progress over time), and **ZPD Radial Chart**.

---

# 7. Page-by-Page Technical Walkthrough

---

## A. `/onboarding` (Psychometric Discovery)

- **Logic:** A state-driven carousel capturing demographics and work history. Redirects to `/path-selection` or `/learn` based on user status.

## B. `/path-selection` (Opportunity Engine)

- **Logic:** Streams career options based on the onboarding profile. "Commit" triggers `selectPath()` to initiate the diagnostic flow.

## C. `/pre-test` (Baseline Initialization)

- **Logic:** 10 questions with **Weighted Scoring** (easy: 1, medium: 2, hard: 3). Submits results to `submitPreTestResults` to set the initial mastery state.

## D. `/learn` (Adaptive Classroom)

- **Logic:** Renders the dynamic Roadmap. Modules are unlocked/locked based on real-time BKT updates from the `QuizModal`.

## E. `/dashboard` (Mastery Analytics)

- **Logic:** Visualizes mastery probabilities and calculates **NLG** (Normalized Learning Gain/Hake Gain) to prove learning impact.

## F. `/architecture` (Systems Overview)

- **Logic:** The infinite canvas showcasing the system architecture. Driven by `SIDE_PANEL_DATA` and the viewport-aware camera.

---

# 8. AI & Streaming Integration

- **Structured Outputs:** Uses Zod schemas to ensure AI returns valid JSON.
- **Fallback UI:** Skeletons are used during the "Thinking" phase; automatic retries are handled in the background.

---

# 9. Development Guidelines

- **File Structure:** Logic in `[route]Client.tsx`, Server hydration in `page.tsx`.
- **API Strategy:** Server Actions for mutations; AI SDK for generation.
- **Accessibility:** All nodes must have `aria-label` identifying their status.